

UART Transmitter / Receiver

SystemVerilog Design, Simulation & Debugging

Digital Control Design Project

Design and implementation of a serial UART transmitter and receiver

Project Report

Designed by: Mohamed Abdelhamid Ahmed Ali

Email: trapcloud8@gmail.com

Supervised by: Eng. Moaz Khaled

Course: Digital Design (FPGA / SystemVerilog)

Tool: ModelSim - Intel FPGA Edition

Abstract

This project designs a serial UART transmitter and receiver in SystemVerilog. The transmitter (TX) drives data on the serial line while the receiver (RX) captures it in a loop connection. The design was verified against an automated grading testbench. Timing problems were found and fixed until all tests (66 of 66) passed.

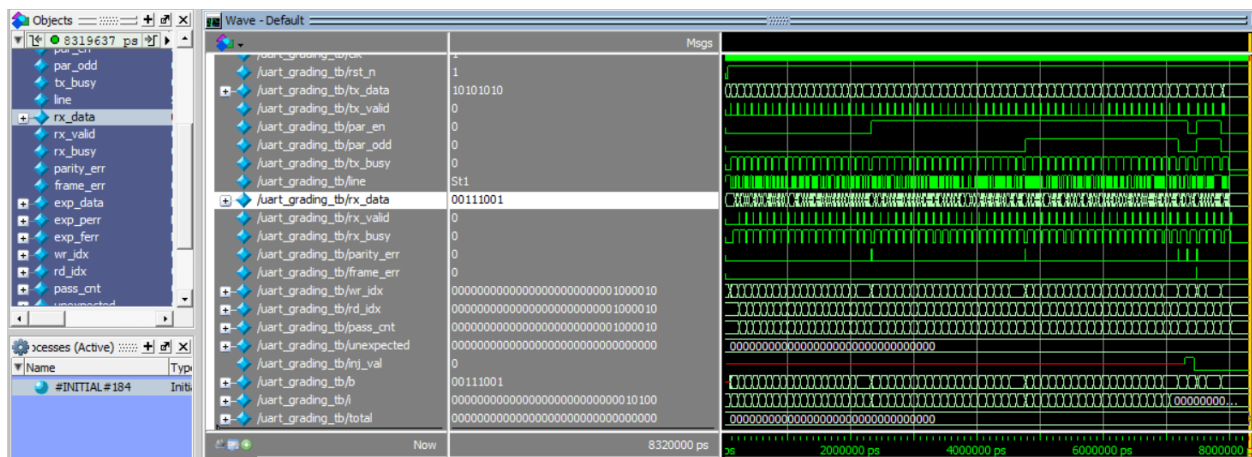
Simulation Results

The design was compiled and simulated with ModelSim. The automated grading testbench sends 66 frames and compares the data against the expected parity error (Parity) and framing error (Framing) flags.

Screenshot: Full pass (66 / 66)

```
Transcript
# ok[65]: d=0x39 pe=0 fe=0
#
# frames sent      : 66
# frames delivered : 66
# unexpected frames: 0
# SCORE 66 66
# GRADING: 66 / 66 frames correct
# RESULT: PASS
# ** Note: $finish      : D:/Work/FPGA Course/SV/Final_Project/uart_loopback_tb.sv(279)
#      Time: 8320 ns   Iteration: 1   Instance: /uart_grading_tb
```

Waveform of the running simulation



Final result: project passed with a score of 66/66

After the fixes, all sixty-six test frames (66/66) passed and the result changed to PASS instead of the earlier FAIL with 64/66.

Module Structure

The project is split into two main files: the transmitter and the receiver. The following explains the internal module structure, with special focus on the receiver.

Transmitter (uart_tx.sv)

The transmitter consists of three internal modules:

uart_fsm - control state machine. It watches the `buffer_ready` signal and produces the serial start and the frame data. Its outputs were made combinational so that transmission begins with no extra delay.

uart_paritycalc / FULLADDER - computes the parity bit (even or odd) from the eight data bits using a chain of full adders.

uart_serializer - the 11-bit shift register driven by the system clock. It outputs the start bit, the eight data bits, the parity slot and the stop slot, one per clock cycle; it also sets the `done` flag when the frame ends.

Receiver (uart_rx.sv)

The receiver packs several modules that work together:

- **edgeDetector** - samples the incoming serial line and produces the meta-stable-resolved, delayed copy plus the detected rising, falling and any edges.
- **sampler** - waits for a falling edge (the start bit), then captures the line state one sample per clock into an 11-bit register. When all 11 bits are stored it raises `ready` and drops `busy`.
- **errorCheckerModule** - combines the sampler output with the parity configuration: it extracts the eight data bits and flags a parity error or a framing error as required by the protocol.
- **uart_rx_fsm** - the receiving control state machine with states IDLE, DATA, PARITY and STOP; it also counts the eight data bits.
- **uart_rx** - the top wrapper that instantiates `errorCheckerModule` and connects the serial input to the parallel output and the two error flags.

Problems Fixed

1. The baud rate generator was removed from the TX module. The pulse is now sent directly on the system clock with no frequency divider, so one bit is transmitted every clock cycle.
2. There was a large delay (5 clocks) in the TX state machine state, which made the start of the frame appear late on the line; the result was 64/66 and the project failed tests 4 and 5 (the parity- and framing-error tests). After shortening the state machine and making its signals combinational, transmission starts immediately and all tests passed 66/66.